Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance

Stephen M. Blackburn, Kathryn S. McKinley

Szabó Márk

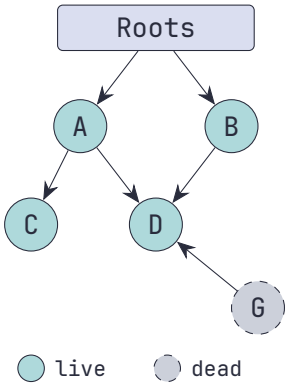
Garbage Collection Background

Tracing Garbage Collection

A tracing collector has three jobs:

- **Allocation** of new objects.
- **Identification**: mark every object reachable from the roots.
- **Reclamation** of unmarked objects.

An object is `dead` if we can't reach it from roots.



The Canonical Tracing Collectors

The paper compares immix against the three canonical tracing collectors:

- **Mark-sweep (MS):** non-moving, objects stay where they were allocated.
- **Semi-space (SS):** moving, copies live objects into a second space.
- **Mark-compact (MC):** moving, slides live objects together within the same space.

Building blocks

More sophisticated collectors, such as generational ones, are built by combining these.

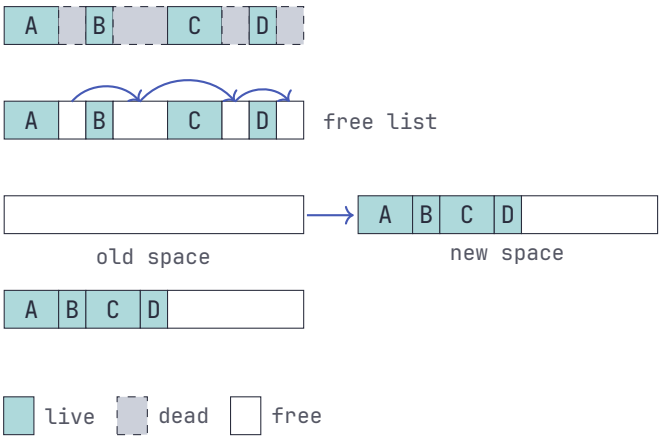
Three Reclamation Strategies

After marking
live and dead objects

Mark-sweep
objects stay in place

Semi-space
evacuate to a new space

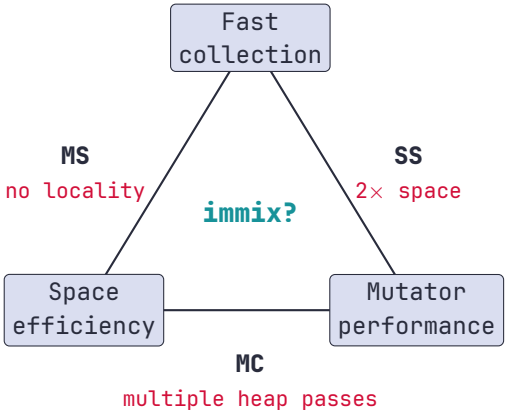
Mark-compact
slide to one end



Three Performance Objectives

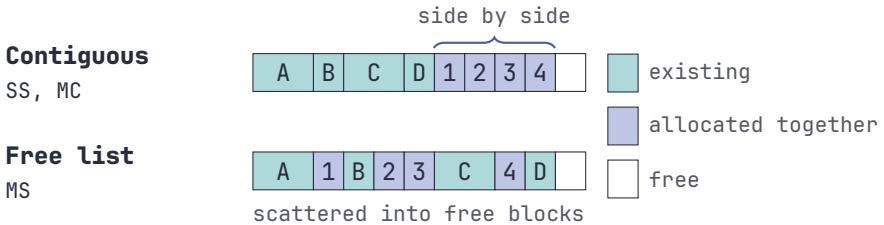
- **Space efficiency:** needs only a small heap.
- **Fast collection:** little time spent collecting.
- **Mutator performance:** the application (*mutator*) runs fast, thanks to locality.

Each canonical collector achieves only two of the three.



Locality and Contiguous Allocation

- **Contiguous allocation** puts objects allocated at the same time next to each other in space: good **mutator locality**.
- **Mark-sweep** is non-moving and allocates from a free list: no locality for objects allocated at the same time.

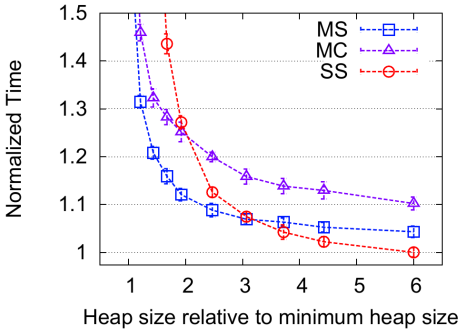


The Space-Time Trade-off

MS, SS and MC from 1× to 6× the minimum heap:

- **Small heap:** GC runs often, fast collection wins (**MS**).
- **Large heap:** GC runs rarely, fast program wins (**SS**).
- **MC:** collecting is too slow.

The crossing lines are the classic **space-time trade-off**.

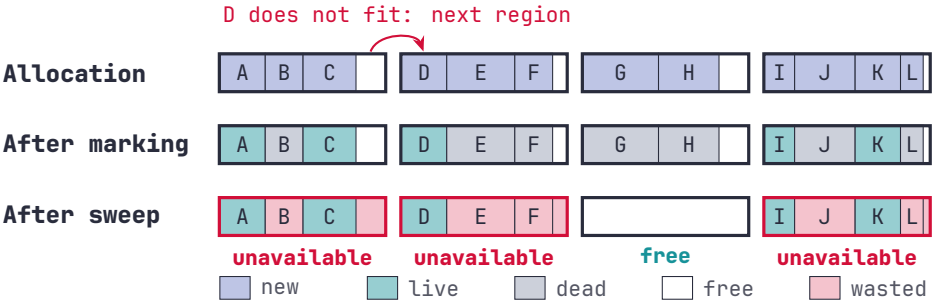


(a) Total Time

Mark-Region Collection

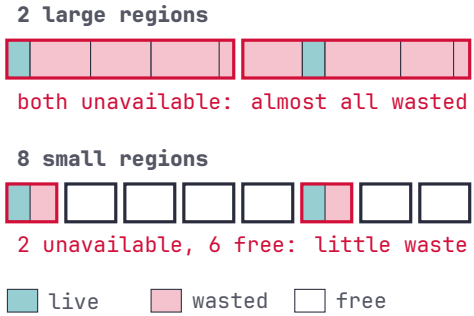
Mark-Region Collectors

- Memory is split into **fixed-size regions**. Objects can't span them.
- **Bump allocation**: objects go one after another, if one does not fit, it goes to the **next free region**.
- A region with any live object stays *unavailable*.



Two Open Questions

1. **How big should regions be?**
   - **Large:** one small object wastes a whole region.
   - **Small:** less waste, but slower collection and unable to allocate big objects.
2. **How to defragment?**
   - A region with **any** live object stays unavailable.

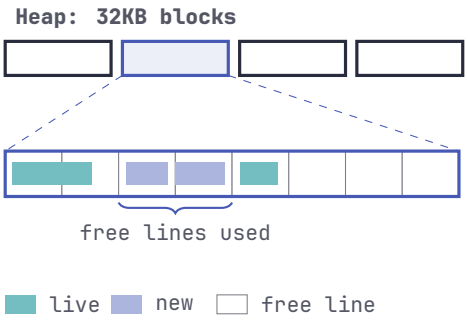


Immix

Blocks (Regions) and Lines

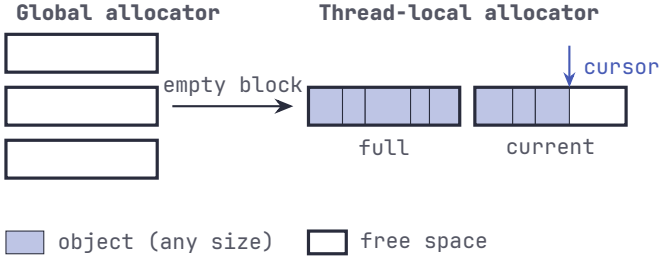
Answers question 1: use **both** sizes. Big **blocks** (regions) keep allocation fast and contiguous. Small **lines** (used for marking) inside them keep waste low.

- 32KB **blocks**, split into 128B **lines**.
- Free block: reclaimed **whole**.
- Partly used block: new objects fill its **free lines**.
- Objects span lines, never blocks.



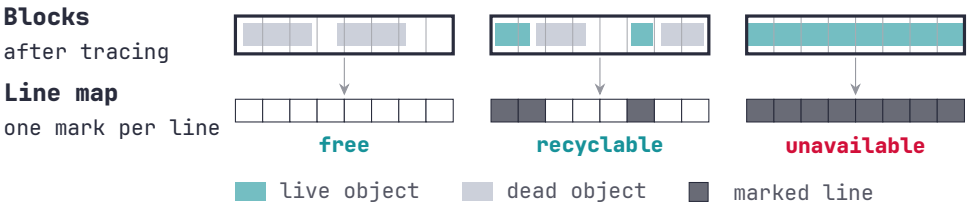
Initial Allocation

- All blocks start **empty**.
- Each thread's allocator takes an empty block from the **global allocator** and bump allocates into it.
- Block full: take another. No empty blocks left: the heap is full, so a collection starts.



Identification and Reclamation

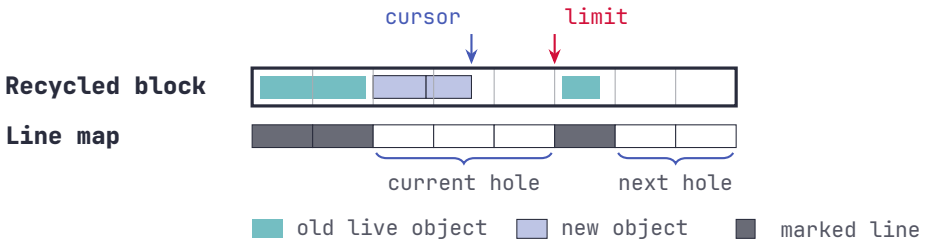
1. **Identification:** trace from the roots and mark each live object **and its lines** in a **line map**.



2. **Reclamation:** read the **line map** and find the entirely free and partially free blocks. Free blocks return to the **global pool** and partially free blocks are **recycled** for the next allocation.

Steady State Allocation

- Allocate into **recycled blocks** first, in address order. Empty blocks come last.
- Scan the line map for a **hole** (one or more free lines in a row) and bump allocate into it.
- **Limit:** the next marked line or the block's end. Past it: find the next hole, or take a new block.



Recycling and Allocation Policies

- **Recycling:** a block with at least `F` free lines is recyclable. Simplest `F = 1` worked best.
- **Allocation:** recyclable blocks first, in address order **free blocks** last.



Optimizations

Exact Line Marking

- Every line a live object lies on must be marked, or its space could be reused.
- **Exact marking**, for each live object:
  1. read its type to find its **size**,
  2. loop over its memory, marking **each line**.
- **Mark-sweep** just sets **one bit** in the header compared to line marking which is expensive.

Key observation

Most objects are smaller than a line (128B), so they span **at most two lines**.

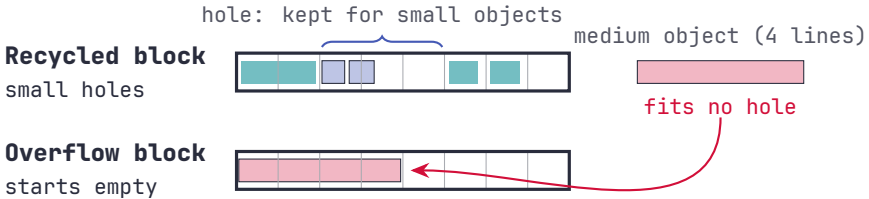
Conservative Line Marking

- **Small object** (up to a line): mark only its **start line**. No size lookup, one write.
- Its possible second line: the allocator **skips the first line** of every hole.
- **Medium object** (larger than a line): marked exactly, but rare.
- One **header bit**, set at allocation, says small or medium.



Demand-Driven Overflow Allocation

- **Problem:** a medium object (larger than a line) may fit no hole of a recycled block, so the allocator skips and wastes many holes.
- **Fix:** each allocator also has an **overflow allocator** that only uses empty blocks.



Opportunistic Defragmentation

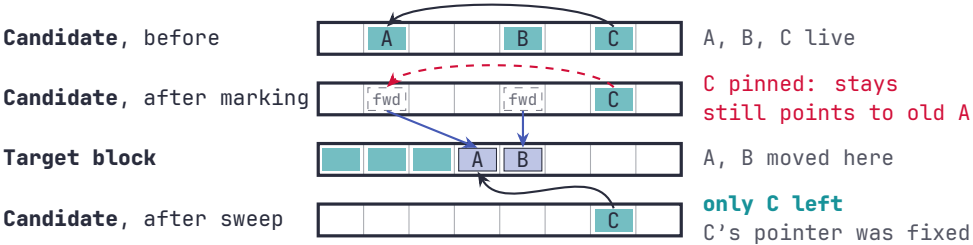
Opportunistic Evacuation

Answers question 2: move objects out of fragmented blocks while marking, only when needed.

- **When:** at the start of a collection, if last time some recyclable blocks went **unused** or too little space was freed.
- **What:** pick the most fragmented blocks as **candidates**, while marking, move their live objects into free space.
- **Opportunistic:** out of space, or the object is pinned? Just mark it in place (plain mark-region behaviour).

Forwarding Pointers and Pinning

- **Forwarding pointer:** points from the old place to the new one. While tracing, references to the old address are fixed. Afterwards the old place is **freed**.
- **Pinned object:** the program asked for it never to move (e.g. an I/O buffer). A header bit marks it.



Candidate Selection

- **More holes = more fragmented**: picked first.
- Walk from the most holes down: their **marked lines** add to the space **needed**, their **free lines** stop counting as **available**. Stop when needed > available.

Each row adds up **all blocks** with that many holes:

Group	Marked	Free	Needed	Available	
start			0	100	
6 holes	10	15	10	85	candidate
5 holes	20	20	30	65	candidate
4 holes	30	25	60	40	stop

→ **all** blocks with 6 or 5 holes become candidates.

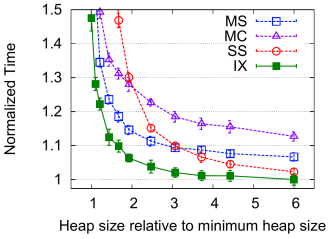
Evaluation

Methodology

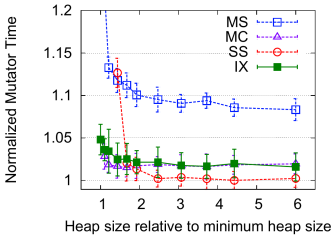
- **Platform:** implemented in **Jikes RVM**, a Java VM, using its memory management toolkit **MMTk**. MS, SS and MC come from the same toolkit.
- **Benchmarks:** 20 standard Java benchmark programs.
- **Hardware:** 3 different machines.
- **Heap sizes:** 1× to 6× the smallest heap mark-sweep can run in.
- **Runs:** each experiment 6 times. Drop the fastest and slowest, report the mean of the other 4.

Full Heap Performance

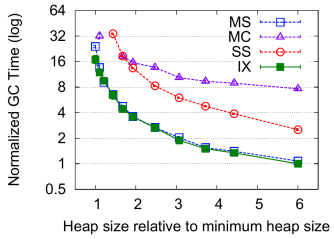
- **(a) Total:** IX is **lowest at every heap size**, typically 5–10% faster.
- **(d) Mutator:** IX is as fast as SS, thanks to locality.
- **(f) Collector:** IX collects as fast as MS.



(a) Total Time Intel Core 2 Duo



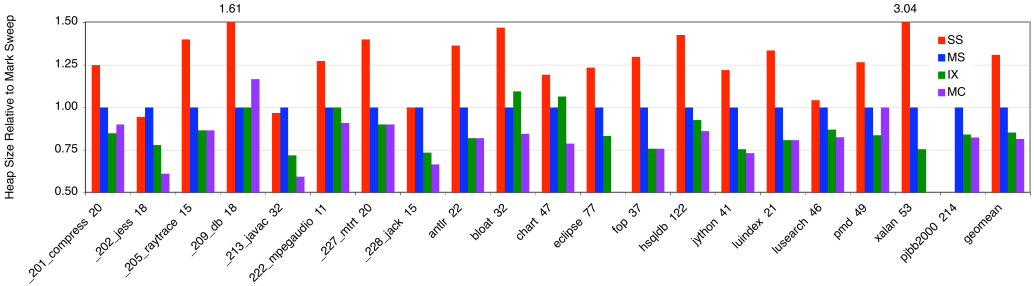
(d) Mutator Time Core 2 Duo



(f) Collector Time Core 2 Duo

Space Efficiency

- Measured as the **smallest heap** each collector can run in, relative to MS. **Lower is better.**
- On average (rightmost group), IX needs **86%** of MS's heap, close to MC with **83%**.
- SS needs the most, because of its copy reserve.



Where the Space Goes

IX vs MS: less space

- MS never moves objects: no defragmentation, so fragmentation stays. This is the **main reason**.
- IX packs objects of any size into holes, and **defragmentation** repairs fragmented blocks.

IX vs MC: slightly more

- MC squeezes **all** objects together every collection: no waste, but slow.
- IX keeps small waste: only **whole lines** are reused, one line is skipped per hole.
- Defragmentation is **partial**: only when needed, only on candidates.

Conclusion

Related Work

Mark-region collectors existed, in JRockit and IBM's JVM, but were never described or evaluated in a publication.

How immix differs:

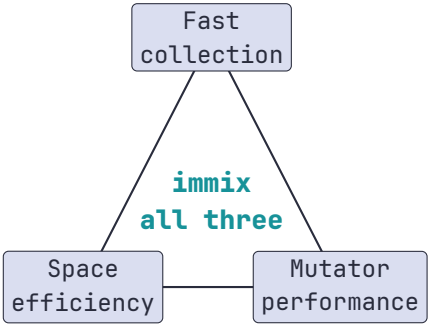
- **Defragmentation:** JRockit and IBM use compaction, immix evacuates opportunistically in a **single pass**.
- **Granularity:** IBM never reuses gaps smaller than 512 bytes, immix reclaims **128-byte lines**.
- **Metadata:** 0.8% of the heap for immix vs 3.1% for IBM.

Why Immix Gets All Three

- **Mutator performance** ← **bump allocation into blocks and holes**: objects allocated together sit together, like SS.
- **Fast collection** ← **mostly non-moving**: mark, then read the line map, like MS. Conservative marking keeps it cheap, defragmentation runs only on demand, in the same pass.
- **Space efficiency** ← **lines** reuse partly used blocks, **defragmentation** frees fragmented ones, and there is no 2× copy reserve.

Conclusion

- **Immix:** mark-region with **blocks and lines**, plus **opportunistic defragmentation** that moves objects only when needed.
- **Faster than all canonical collectors** at every heap size, typically by **5–10%**.



References

- Stephen M. Blackburn, Kathryn S. McKinley. *Immix: A Mark-Region Garbage Collector with Space Efficiency, Fast Collection, and Mutator Performance*. PLDI, 2008.